A (More) Objective View of AI

Software Development in the Age of AI

Limitations and Engineering Practices

Federico Busato

2026-06-23

1. Limitations of AI-Generated Code

The Slot Machine and the Illusion of Control 💬

> *"The thing about AI based coding is that it's like a **slot machine**, and that you have an **illusion of control**, you know, you can get to craft your prompt, and your list of MCPs, and your skills, and whatever, and then in the end, you pull the lever. Right?*
>
> *... **It's the stochastic thing. You get the occasional win. It's like, oh, I won. I got a feature.**"* [1]

— **The Dangerous Illusion of AI Coding?**,
Jeremy Howard, 2026

[1] The Dangerous Illusion of AI Coding? - Jeremy Howard, 2026

Generalization 💬

- **Lack of generalization beyond the training data.**

> *"This brittleness means that* ***coding agents are almost frighteningly good at what they've been trained and fine-tuned on—modern programming languages****, JavaScript, HTML, and similar well-represented technologies—and generally* ***terrible at tasks on which they have not been deeply trained****.*
>
> *... It took me five minutes to make a nice HTML5 demo with Claude but a week of torturous trial and error, plus actual systematic design on my part, to make a similar demo of an Atari 800 game."* [2]
>
> — **Benj Edwards**,
> Ars Technica

[2] 10 things I learned from burning myself out with AI coding agents - Benj Edwards, Ars Technica, 2026

Creativity 💬

"Due to what might poetically be called "preconceived notions" baked into a coding model's neural network (more technically, statistical semantic associations), ***it can be difficult to get AI agents to create truly novel things****, even if you carefully spell out what you want."* [2]

— **Benj Edwards**,
Ars Technica

- **Bug proliferation**

> *"**Fixing bugs can also create bugs elsewhere. Coding agents supercharge this phenomenon** because they can barrel through your code and make sweeping changes in pursuit of narrow-minded goals that affect lots of working systems."* [2]
>
> — **Benj Edwards**, Ars Technica

> *"A comprehensive report analyzed 470 real-world open-source pull requests and found that **AI-generated code introduces 1.7x more defects** across every major category of software quality—including logic, maintainability, security, and performance."* [3]
>
> — **CodeRabbit**

[3] State of the AI vs. Human Code Generation Report - CodeRabbit, 2025

*"**LLMs cosplay understanding things.***

*... **The difference between pretending to be intelligent and actually being intelligent is entirely unimportant, as long as you're in the region in which the pretense is actually effective**, you know. So it's actually fine for a great many tasks that LLMs only pretend to be intelligent, because for all intents and purposes, it just doesn't matter **until you get to the point where it can't pretend anymore**. And then you realize, like, oh my god. This thing is so stupid."* [1]

— **Jeremy Howard**,
The Dangerous Illusion of AI Coding?, 2026

"61% agree that "AI often produces code that looks correct but isn't reliable." That's a critical finding—it means ***AI code can introduce subtle bugs that are harder to spot than typical human errors.***

*The same percentage (61%) agree that it "****requires a lot of effort to get good code from AI****" through prompting and fixing."* [4]

— **Sonar**,
State of Code Developer Survey report, 2026

[4] State of Code Developer Survey report - Sonar, 2026

"Overall performance scores drop significantly from $> 80\%$ on isolated tasks to at most 38% in continuous settings, ***exposing agents' profound struggle with longterm maintenance and error propagation****.* [5]

— **EvoClaw: Evaluating AI Agents on Continuous Software Evolution**, Deng et al., 2026

[5] EvoClaw: Evaluating AI Agents on Continuous Software Evolution - Deng et al., arXiv, June 2026

1. ***Context drift***. *As codebases grow beyond the effective context window, agents lose coherent understanding of system-wide invariants and dependencies.*
2. ***Error propagation***. *A small error in an early commit cascades into compounding failures in subsequent work, and agents lack robust mechanisms for detecting and recovering from these chains.*
3. ***Technical debt awareness***. *Agents do not currently model the long-term costs of their design decisions — they optimize for immediate task completion without considering maintainability.*
4. ***Verification fidelity***. *Automated testing remains incomplete; agents can pass tests while introducing subtle semantic errors that only manifest under novel inputs.* [6]

— **Agentic Software: How AI Agents Are Restructuring the Software Paradigm**, Z. Cao, 2026

[6] Agentic Software: How AI Agents Are Restructuring the Software Paradigm - Z. Cao, arXiv, June 2026

- **The “Last Mile” Problem**. Prototyping with AI is not the same as building a production-quality product.

> *“The first 90 percent of an AI coding project comes in fast and amazes you.* ***The last 10 percent involves tediously filling in the details*** *through back-and-forth trial-and-error conversation with the agent.”* [2]
>
> — **Benj Edwards**,
> Ars Technica, 2026

“Honest reflections from coding with AI so far as a non-engineer:

It can get you 70% of the way there, but that ***last 30% is frustrating****. It keeps taking one step forward and two steps backward with new bugs, issues, etc.*

If I knew how the code worked I could probably fix it myself. But since I don’t, ***I question if I’m actually learning that much.****“* [7]

— Peter Yang

[7] Peter Yang

With AI, it's easier to get the first 90 percent out there. This means we can spend more time on the remaining 10 percent, which means more time for craftsmanship and figuring out how to make your users happy. [8]

— **The 100 hour gap between a vibecoded prototype and a working product**, M. Budkowski, 2026

[8] The 100 hour gap between a vibecoded prototype and a working product - M. Budkowski, 2026

2. AI as a Software Engineering Tool

- **LLMs should not be seen as a substitute for software engineering.**

> ***Software engineering is an unusual discipline, and a lot of people mistake it for being the same as typing code into an IDE.***
>
> *... Because software engineering is all about finding what those pieces are, and how they should behave, and then how you can put them together to create a bigger piece, and then how you can put them together to create a bigger piece.* ***And if we do that well, then in 10 years' time, we could have software that is far more capable than anything we could even imagine today.***" [9]
>
> — **Jeremy Howard**,
> The Dangerous Illusion of AI Coding?, 2026

[9] The Dangerous Illusion of AI Coding? - Jeremy Howard, 2026

"In ProgramBench, given only a program and its documentation, agents must architect and implement a codebase that matches the reference.

... Our 200 tasks range from compact CLI tools to widely used software such as FFmpeg, SQLite, and the PHP interpreter. We evaluate 9 LMs and find that ***none fully resolve any task****.*

... Models favor monolithic, single-file implementations ***that diverge sharply from human-written code****."* [10]

— **ProgramBench: Can Language Models Rebuild Programs From Scratch?**, Yang et al., 2026

[10] ProgramBench: Can Language Models Rebuild Programs From Scratch? - Yang et al., ArXiv, May 2026

*"**One of the lesser spoken truths of working productively with LLMs as a software engineer on non-toy-projects is that it's difficult**. There's a lot of depth to understanding how to use the tools, there are plenty of traps to avoid, and the pace at which they can churn out working code raises the bar for what the human participant can and should be contributing.*

*... Iterating with coding agents to **produce production-quality** code that I'm confident **I can maintain in the future** feels like a different process entirely."* [11]

— Simon Willison,
Vibe engineering

[11] Vibe engineering - Simon Willison, 2025

*"I'm not very happy with the code quality and I think **agents bloat abstractions**, have poor code aesthetics, are very prone to copy pasting code blocks and **it's a mess**, but at this point I stopped fighting it too hard and just moved on."* [12]

— **Andrej Karpathy**

[12] Andrej Karpathy, 2026

*"**Programming, like writing, is an activity, where one iteratively sharpens what they're doing as they do it.***

*... But, **vibe coding gives the illusion that your vibes are precise abstractions**. They will feel this way right up until they leak, which will happen when you add enough features or get enough scale. **Unexpected behaviors (bugs) that emerge from lower levels of abstraction that you don't understand** will sneak up on you and wreck your whole day."* [13]

— **Reports of code's death are greatly exaggerated**,
Steve Krouse

[13] Reports of code's death are greatly exaggerated - Steve Krouse, 2026

*"**LLMs** have effectively killed the cost of generating lines of code, but they **haven't touched the cost of truly understanding a problem**. We're seeing a flood of "apps built in a weekend," but most of these are just thin wrappers around third-party APIs. They look impressive in a Twitter demo, but they often crumble the moment they hit the friction of the real world...*

*... In this new reality, **engineering expertise remains incredibly valuable**, but the nature of the role is shifting. Relevance is not fading. Instead, it is about leveraging these tools to build at a higher level than was previously possible. True expertise is now required to steer these systems and provide the technical oversight that LLMs currently lack."* [14]

— Code Is Cheap Now. Software Isn't,
Chris Gregori

[14] Code Is Cheap Now. Software Isn't - Chris Gregori, 2026

"Creating durable production code, managing a complex project, or crafting something truly novel still requires experience, patience, and skill beyond what today's AI agents can provide on their own.

... veteran software developers probably shouldn't fear losing their jobs to these tools any time soon. ***In fact, they may become busier than ever****.*

... I don't think AI tools will make human software designers obsolete. Instead, they may well help those designers become more capable." [2]

— **Benj Edwards**,
Ars Technica

- **AI systems complement human expertise. They do not substitute for it.**

> *"What the models do, the capabilities they have, and the quality of the content they produce is, to a large extent,* ***a reflection of the user****."* [15]
>
> — **The skeptic's guide to generative AI assisted coding**, Rob Patro

[15] The skeptic's guide to generative AI assisted coding - Rob Patro, combine-lab, 2026

*"**Experienced human software developers bring judgment, creativity, and domain knowledge that AI models lack**. They know how to architect systems for long-term maintainability, how to balance technical debt against feature velocity, and when to push back when requirements don't make sense.*

*... They can automate many tasks, but **managing the overarching project scope still falls to the person telling the tool what to do**."* [2]

— **Benj Edwards**,
Ars Technica, 2026

"Expertise has always been marked by a deep knowledge of software qualities and how they are achieved through implementation; ***understanding architectural complexity****; capacity to continuously learn and change practices; providing credible, honest, trustworthy information, and a long tail of other soft skills.*

... As today's frontier models and their successors take over the keyboard, the human engineer's most important tool is no longer the integrated development environment; ***it is their judgment.*** *"* [16]

— **The End of the Coder?**,
Communications of the ACM, 2026

[16] The End of the Coder? - L. Kugler, Communications of the ACM, 2026

*"Knowing something about how good software development works helps a lot when guiding an **AI coding agent—the tool amplifies your existing knowledge rather than replacing it**."* [2]

— **Benj Edwards**, Ars Technica

*"**AI tools amplify existing expertise**. The more skills and experience you have as a software engineer the faster and better the results you can get from working with LLMs and coding agents."* [11]

— **Simon Willison**, Vibe engineering

"Relying too much on AI risks missing subtle bugs, architectural flaws, and vulnerabilities that only skilled engineers can catch. ***Human oversight, critical thinking, and domain knowledge are indispensable*** *for both correcting errors and driving innovation as technology progresses.*

Generative AI currently acts as seniority-biased technological change: ***It disproportionately amplifies engineers who already possess systems judgment****, like a taste for architecture, debugging under uncertainty, and operational intuition."* [17]

— **Redefining the Software Engineering Profession for AI**,
Communications of the ACM, 2026

[17] Redefining the Software Engineering Profession for AI - Russinovich & Hanselman, Communications of the ACM, 2026

"LLMs have evolved into enormously powerful engines that can produce vast amounts of artifacts. However, ***they need to be guided to produce output that is actually valuable****. Without this, they can produce a mountain of garbage just as easily as gold.*

... The fluency of the AI makes it easy to think you should interact with it like you would a junior engineer. ***The best output comes from realizing that it is a machine that produces code and executes tasks****, and you should treat it as such."* [18]

— **'AI makes me 10x productive', 'AI produces garbage'**,
Michael Rothrock, 2026

[18] 'AI makes me 10x productive', 'AI produces garbage' - M. Rothrock, 2026

"When I was working on something I already understood deeply, ***AI was excellent****. I could review its output instantly, catch mistakes before they landed and move at a pace I'd never have managed alone.*

... When I was working on something I could describe but didn't yet know, ***AI was good but required more care****.*

... When I was working on something where I didn't even know what I wanted, ***AI was somewhere between unhelpful and harmful****.*

... But expertise alone isn't enough. Even when I understood a problem deeply, ***AI still struggled if the task had no objectively checkable answer*** [19]

— **Eight years of wanting, three months of building with AI**,
L. Maganti, 2026

[19] Eight years of wanting, three months of building with AI - L. Maganti, 2026

Plausibility over Correctness 💬

*"**LLMs optimize for plausibility over correctness**. In this case, plausible is about 20,000 times slower than correct.*

*... THIS is the failure mode. Not broken syntax or missing semicolons. The code is syntactically and semantically correct. **It does what was asked for. It just does not do what the situation requires.**"* [20]

— **Your LLM Doesn't Write Correct Code. It Writes Plausible Code.**, Hōrōshi, 2026

[20] Your LLM Doesn't Write Correct Code. It Writes Plausible Code. - Hōrōshi, 2026

3. Engineering Practices in the Age of AI

LLMs reward existing software engineering practices. The following list presents a concise set of practices through which AI systems can enhance the software development process. This list has been compiled from engineers' public notes, discussions with colleagues, and personal experience.

- **Code Review**. LLMs rely on pattern matching and memorization. They can be very effective at finding common issues, faulty logic, or missing assumptions in code. *This is not a substitute for human code review*.
- **Documentation**. Writing documentation is essential for understanding and maintaining software, but it is often a time-consuming and boring task. LLMs can help write documentation, or a draft, quickly, *to review and refine later*.
- **Code Explanation**. LLMs can parse code much faster than humans. As a result, they can be very useful for explaining code in context or for understanding the organization and workflow of large codebases.

- **Prototyping**. Productization is the most demanding phase of the development process, and it requires a clear idea of what to achieve, which is often not the case. LLMs can be very effective at quickly implementing new features or ideas, *even if the quality is far from production-ready*.

- **Discussing Ideas and Intuitions**. It is common to reflect on new ideas or revisit earlier ones when thinking through specific aspects of a problem. LLMs can discuss these topics much as a coworker might, even before prototyping.

- **Evaluating Alternatives**. There are dozens of ways to solve a given problem. LLMs can suggest alternatives that users can evaluate and then select the most suitable option *depending on the context*.

- **Enforcing Coding Style**. Large codebases involving several engineers tend to develop high-level practices that are often difficult to enforce with common tools. Coding style can fall into this category. Requirements such as "the code shall not use lambda expressions" are difficult to enforce with traditional tools but trivial for LLMs.

- **Testing**. Robust and comprehensive test suites prevent users *and LLMs* from unintentionally introducing bugs. Additionally, LLMs can quickly draft tests that can be adjusted later.

- **Refactoring**. Technology evolves faster than engineers can keep up with, especially in projects with limited engineering resources. LLMs are an excellent tool for modernizing codebases, but it is important to pay attention to *maintaining the underlying logic*.

- **Debugging**. LLMs can automate the debugging process, especially for simple bugs. They can compile, execute, analyze the output, or even navigate the `git` history and find connections in the code to identify problems or formulate hypotheses.

- **Vibe Coding Tools**. It is common to face limitations with open-source tools because each project has its own specific context. Engineering work often does not leave time to contribute to external projects to overcome these limitations. LLMs can implement small features while engineers stay focused on the actual code. One example could be adding a new check to `clang-tidy`.

- **Help with Unfamiliar Tasks**. It is common to work on projects outside area of expertise. For example, LLMs can assist by explaining new syntax and translating between languages.

- **Git Interaction**. LLMs show excellent capabilities when working with `git`. They can navigate history, reverse changes, and identify the root causes of bugs. They can also handle complex rebases without human intervention.